Formative Assignment: Advanced Linear Algebra (PCA)

This notebook will guide you through the implementation of Principal Component Analysis (PCA). Fill in the missing code and provide the required answers in the appropriate sections. You will work with a dataset that is Africanized
.

1. Make sure to display outputs for each code cell when submitting.
2. Do not write all your code on one cell
3. Do not use any libraries aside from numpy

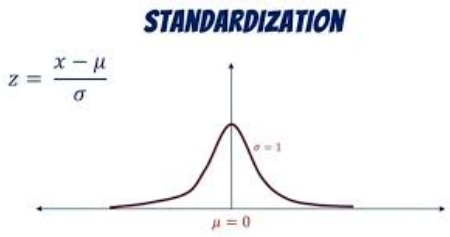
Link to our github:https://github.com/TRolande/Principle_Component_Analysis.git

```
from google.colab import drive
drive.mount('/content/drive')
```

Step 1: Load and Standardize the Data

Before applying PCA, we must standardize the dataset. Standardization ensures that all features have a mean of 0 and a standard deviation of 1, which is essential for PCA. Fill in the code to standardize the dataset.

STRICTLY - Write code that implements standardization based on the image below




```
# Step 1: Load and Standardize the data (use of numpy only allowed)
import numpy as np
import csv

with open('Africa_Life_Expectancy_Data.csv', newline='') as f:
    reader = csv.reader(f)
    rows = list(reader)

raw_data = rows[1:]
status_map = {'Developing': 0, 'Developed': 1}

numeric_data = []
for row in raw_data:
    try:
        encoded_row = [
            float(row[1]),
            float(status_map.get(row[2], np.nan)),
        ]
        for val in row[3:]:
            encoded_row.append(float(val) if val.strip() != '' else np.nan)
        numeric_data.append(encoded_row)
    except:
        continue

data_array = np.array(numeric_data, dtype=float)
col_means = np.nanmean(data_array, axis=0)
inds = np.where(np.isnan(data_array))
data_array[inds] = np.take(col_means, inds[1])

mean = np.mean(data_array, axis=0)
std = np.std(data_array, axis=0)
std[std == 0] = 1
```

```
standardized_data = (data_array - mean) / std

standardized_data[:5]
```

```
array([[ 1.62697843e+00,  0.00000000e+00,  2.12078989e+00,
        -1.58247157e+00, -2.88676362e-01, -1.31656136e-15,
        -5.08760734e-01,  8.67824489e-01, -2.46410042e-01,
         2.90225851e+00, -3.47360169e-01,  8.95477286e-01,
        -3.09862638e-15,  8.86410525e-01, -6.45511261e-01,
         1.04321429e+00,  1.79142628e+00, -4.73169023e-01,
        -5.13700318e-01,  1.79265982e+00,  1.83196233e+00],
       [ 1.41004798e+00,  0.00000000e+00,  2.09582191e+00,
        -1.63360734e+00, -2.88676362e-01, -9.79088572e-01,
        -2.54858878e-01,  8.67824489e-01, -2.50253502e-01,
         2.81437821e+00, -3.47360169e-01,  8.95477286e-01,
         9.12680924e-01,  8.86410525e-01, -6.45511261e-01,
        -4.07746356e-01,  1.74807014e+00, -4.73169023e-01,
        -5.13700318e-01,  1.77987349e+00,  1.83196233e+00],
       [ 1.19311752e+00,  0.00000000e+00,  2.08333792e+00,
        -9.88018243e-01, -2.88676362e-01, -7.86387451e-01,
         2.03998354e+00,  8.67824489e-01, -2.48728320e-01,
         2.71850879e+00, -3.47360169e-01,  8.95477286e-01,
         8.67825655e-01,  8.86410525e-01, -6.45511261e-01,
         1.58520455e+00,  1.70376845e+00, -5.10776649e-01,
        -5.13700318e-01,  1.75430082e+00,  1.83196233e+00],
       [ 9.76187060e-01,  0.00000000e+00,  2.05836994e+00,
        -9.81626272e-01, -2.88676362e-01, -7.38212170e-01,
         2.09370319e+00,  8.67824489e-01, -2.49155371e-01,
         2.63062849e+00, -3.47360169e-01,  8.95477286e-01,
         3.79401614e-01,  8.86410525e-01, -6.45511261e-01,
         1.62282883e+00,  1.65958317e+00, -5.10776649e-01,
        -5.13700318e-01,  1.72233499e+00,  1.83196233e+00],
       [ 7.59256602e-01,  0.00000000e+00,  2.03340196e+00,
        -9.62450358e-01, -2.88676362e-01, -7.75270078e-01,
         1.87403707e+00,  8.67824489e-01, -2.43420685e-01,
         2.54274819e+00, -3.47360169e-01,  8.95477286e-01,
        -4.42314836e-02,  8.86410525e-01, -6.45511261e-01,
         1.56917095e+00,  1.61690899e+00, -5.10776649e-01,
        -5.13700318e-01,  1.67118966e+00,  1.69072641e+00]])
```

```
# Step 2: Explore the Data
print("Dataset Shape:", data_array.shape)
print("Number of African Countries:", len(set(row[0] for row in raw_data)))
print()
print("Columns in dataset:")
headers = ['Year', 'Status', 'Life expectancy', 'Adult Mortality', 'Infant Deaths',
           'Alcohol', 'Percentage Expenditure', 'Hepatitis B', 'Measles', 'BMI',
           'Under-five Deaths', 'Polio', 'Total Expenditure', 'Diphtheria',
           'HIV/AIDS', 'GDP', 'Population', 'Thinness 1-19yrs',
           'Thinness 5-9yrs', 'Income Composition', 'Schooling']
for i, h in enumerate(headers):
    print(f"  Column {i}: {h}")
print()
print("Missing values handled per column:")
missing = np.sum(np.isnan(data_array), axis=0)
for i, m in enumerate(missing):
    if m > 0:
        print(f"  {headers[i]}: {m} missing values imputed with column mean")
```

```
Dataset Shape: (864, 21)
Number of African Countries: 54

Columns in dataset:
  Column 0: Year
  Column 1: Status
  Column 2: Life expectancy
  Column 3: Adult Mortality
  Column 4: Infant Deaths
  Column 5: Alcohol
  Column 6: Percentage Expenditure
  Column 7: Hepatitis B
  Column 8: Measles
  Column 9: BMI
  Column 10: Under-five Deaths
  Column 11: Polio
```

```
  Column 12: Total Expenditure
  Column 13: Diphtheria
  Column 14: HIV/AIDS
  Column 15: GDP
  Column 16: Population
  Column 17: Thinness 1-19yrs
  Column 18: Thinness 5-9yrs
  Column 19: Income Composition
  Column 20: Schooling

Missing values handled per column:
```

Step 3: Calculate the Covariance Matrix

The covariance matrix helps us understand how the features are related to each other. It is a key component in PCA.

```
# Step 3: Calculate the Covariance Matrix
cov_matrix = np.cov(standardized_data, rowvar=False)
cov_matrix
```

```
array([[ 1.00115875,  0.        ,  0.33226393, -0.09502719, -0.0438742 ,
        -0.15617436,  0.08896498,  0.02294788, -0.13448126,  0.16828578,
        -0.05928963,  0.16824066,  0.11586274,  0.2118174 , -0.27431862,
         0.12638706,  0.05051549, -0.15254868, -0.14727753,  0.31426884,
         0.27381493],
       [ 0.        ,  0.        ,  0.        ,  0.        ,  0.        ,
         0.        ,  0.        ,  0.        ,  0.        ,  0.        ,
         0.        ,  0.        ,  0.        ,  0.        ,  0.        ,
         0.        ,  0.        ,  0.        ,  0.        ,  0.        ,
         0.        ],
       [ 0.33226393,  0.        ,  1.00115875, -0.49450496, -0.2430302 ,
        -0.20423359,  0.25574068,  0.26018022, -0.15549135,  0.50567906,
        -0.25930481,  0.40635016, -0.11436011,  0.41417323, -0.51814601,
         0.28934684, -0.0165033 , -0.16265493, -0.16080913,  0.58741192,
         0.49607053],
       [-0.09502719,  0.        , -0.49450496,  1.00115875,  0.03300664,
         0.16413219, -0.01522735, -0.13409037, -0.04374157, -0.17572595,
         0.03338755, -0.10927567,  0.09348319, -0.08733432,  0.44882937,
        -0.03218942, -0.00568252,  0.12848749,  0.13914403, -0.15760407,
        -0.11467694],
       [-0.0438742 ,  0.        , -0.2430302 ,  0.03300664,  1.00115875,
         0.21942096, -0.12651318, -0.19792815,  0.52795494, -0.15242115,
         0.99853378, -0.24860445, -0.11463224, -0.27607   , -0.03327791,
        -0.10520194,  0.54884545,  0.17015176,  0.14602393, -0.17871476,
        -0.12679187],
       [-0.15617436,  0.        , -0.20423359,  0.16413219,  0.21942096,
         1.00115875,  0.28355793, -0.04252989,  0.11264316, -0.07780218,
         0.22891205, -0.0490196 ,  0.04959029, -0.02965361,  0.30597884,
         0.28080617,  0.10383721,  0.14177899,  0.17620667,  0.07820724,
         0.16133425],
       [ 0.08896498,  0.        ,  0.25574068, -0.01522735, -0.12651318,
         0.28355793,  1.00115875,  0.12245206, -0.0847377 ,  0.25630496,
        -0.12667113,  0.13041283,  0.04069829,  0.14999997,  0.0831525 ,
         0.86010423, -0.01686642, -0.01107415,  0.04312736,  0.41339081,
         0.38289454],
       [ 0.02294788,  0.        ,  0.26018022, -0.13409037, -0.19792815,
        -0.04252989,  0.12245206,  1.00115875, -0.06972208,  0.18913528,
        -0.19785359,  0.44370537,  0.09435379,  0.61020531, -0.08813988,
         0.11555899, -0.09539666, -0.09747696, -0.0698296 ,  0.15918128,
         0.18102663],
       [-0.13448126,  0.        , -0.15549135, -0.04374157,  0.52795494,
         0.11264316, -0.0847377 , -0.06972208,  1.00115875, -0.11329924,
         0.52869894, -0.1705056 , -0.1013496 , -0.17451524, -0.01925417,
        -0.06853639,  0.24101276,  0.16910576,  0.16471941, -0.17064144,
        -0.08364189],
       [ 0.16828578,  0.        ,  0.50567906, -0.17572595, -0.15242115,
        -0.07780218,  0.25630496,  0.18913528, -0.11329924,  1.00115875,
        -0.16510677,  0.29325383,  0.01451793,  0.29193281, -0.03579772,
         0.29210621,  0.0338417 , -0.18942708, -0.15685064,  0.50613689,
         0.46556317],
       [-0.05928963,  0.        , -0.25930481,  0.03338755,  0.99853378,
         0.22891205, -0.12667113, -0.19785359,  0.52869894, -0.16510677,
         1.00115875, -0.26057198, -0.11123758, -0.28559877, -0.02909455,
```

```
          -0.11089188,  0.54831121,  0.18739777,  0.16189482, -0.19996519,
          -0.14714145],
         [ 0.16824066,  0.        ,  0.40635016, -0.10927567, -0.24860445,
          -0.0490196 ,  0.13041283,  0.44370537, -0.1705056 ,  0.29325383,
          -0.26057198,  1.00115875,  0.18595725,  0.69340445, -0.00598407,
```

In a paragraph that has a maximum if 5 Lines, Explain why we need to compute a covariance matrix, provide atleast 2 reasons

As observed above, the covariance matrix plays crucial role in the PCA for two reasons. First, it exploits the pairwise linear relationships between all the features, for example, whether GDP and Life Expectancy grow together across countries in Africa.Second, it is directly used as input for eigendecomposition, which gives directions of maximum variation in the data. If there is no covariance matrix, there is no way to find out which combinations of features provide the most information, and hence no way to reduce the dimensions. That what we found during this assignment that we were working on.

Step 4: Perform Eigendecomposition

Eigendecomposition of the covariance matrix will give us the eigenvalues and eigenvectors, which are essential for PCA. Fill in the code to compute the eigenvalues and eigenvectors of the covariance matrix.

```
# Step 4: Perform Eigendecomposition
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)

# Take only real parts
eigenvalues = np.real(eigenvalues)
eigenvectors = np.real(eigenvectors)

eigenvalues, eigenvectors
```

```
(array([4.71299096e+00, 2.68323819e+00, 2.28439880e+00, 1.67865482e+00,
        1.54430684e+00, 1.15842601e+00, 9.82404941e-01, 7.99299000e-01,
        7.31607038e-01, 6.90935309e-01, 5.19731570e-01, 4.96273509e-01,
        4.42226418e-01, 4.29487860e-01, 2.05366981e-03, 2.60220067e-01,
        1.12918790e-01, 1.47781288e-01, 1.67447695e-01, 1.78772194e-01,
        0.00000000e+00]),
 array([[ 1.75384455e-01, -5.34429692e-02, -1.95271981e-01,
          -8.91743504e-03,  3.97306688e-02,  4.92213460e-01,
           4.65635936e-01,  2.94215839e-01, -3.04421509e-01,
          -1.50060345e-01, -4.82251169e-02,  1.15476037e-01,
          -2.79979736e-01, -3.94777265e-01,  2.99977232e-03,
          -8.34964870e-02,  3.75775683e-02,  2.58329446e-02,
          -8.05637564e-02, -1.73723213e-02,  0.00000000e+00],
         [ 0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
           0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
           0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
           0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
           0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
           0.00000000e+00,  0.00000000e+00,  1.00000000e+00],
         [ 3.45854431e-01, -9.85553040e-02, -2.68923095e-01,
          -3.51228738e-02, -2.34138281e-01,  1.96582632e-02,
          -3.16781198e-02, -5.84993954e-02,  2.80354163e-02,
           9.00392698e-02,  1.10248719e-01,  4.45997480e-02,
           2.66908196e-01,  8.29065119e-02,  4.96016186e-03,
           1.80300327e-01, -1.93086524e-01,  1.72682281e-01,
          -7.10488789e-01, -1.78623346e-01,  0.00000000e+00],
         [-1.37522136e-01,  2.08234707e-02,  3.65789665e-01,
           5.55328311e-02,  2.49090576e-01,  2.16047301e-01,
          -1.63793715e-02,  5.63148043e-01,  8.08802901e-02,
          -2.25319936e-01, -2.29924641e-02,  2.02924740e-01,
           5.11689661e-01,  2.00192703e-01,  6.15289025e-03,
          -7.01728482e-03, -5.16239456e-04,  1.29499762e-02,
          -1.21342738e-01, -4.17272467e-02,  0.00000000e+00],
         [-2.62888458e-01, -3.77860615e-01, -2.47587191e-01,
           1.38532858e-01,  1.50754615e-01, -7.42923311e-03,
           1.20534785e-02,  6.94036225e-02, -3.25796683e-02,
           4.62514896e-03,  1.38931304e-02,  1.30969910e-01,
          -2.19344376e-01,  3.33189770e-01, -7.03372927e-01,
           5.90805272e-02, -2.72411535e-02,  1.10523484e-02,
```

```
 -5.41564762e-02, -1.72761233e-02,  0.00000000e+00],
[-5.48121242e-02, -2.98484636e-01,  2.85370132e-01,
 -1.33697352e-03,  1.96774805e-01, -1.89377341e-01,
  5.03623164e-02, -1.83694606e-01, -4.11052438e-01,
  4.92406489e-01,  1.33428001e-02,  3.97682352e-01,
  2.14259554e-01, -3.04889375e-01, -8.06764275e-03,
 -3.08186033e-02,  4.16543728e-02,  1.89918243e-02,
 -5.44415383e-02,  3.00619621e-02,  0.00000000e+00],
[ 2.10848685e-01, -3.04647377e-01,  2.42677643e-01,
 -2.54560273e-01,  5.12995758e-02, -2.37227290e-01,
  3.35770822e-01, -1.43688313e-02,  2.04629326e-01,
 -1.37427787e-01,  6.70031280e-02, -1.44357024e-01,
 -7.53020240e-02,  7.49843086e-02, -1.26219978e-02,
  1.15704819e-01,  6.35788433e-01, -1.13641874e-01,
 -1.33659778e-01, -1.45767840e-01,  0.00000000e+00],
[ 2.15205777e-01,  3.02370382e-02,  4.93388355e-03,
  4.13117409e-01, -2.78946276e-02, -4.22638370e-01,
```

Step 5: Sort Principal Components

Sort the eigenvectors based on their corresponding eigenvalues in descending order. The higher the eigenvalue, the more important the eigenvector. Complete the code to sort the eigenvectors and print the sorted components.

How Is Explained Variance Used In PCA?'

we discovered on this assignment that Explained variance is used in PCA to figure out the amount of information each principal component keeps from the original data set. The eigenvalues are the proportions of variance explained by the principal components. Sorting eigen values in the descending order will ensure that PC1 will have the highest variance, PC2 will have the second highest variance etc. The next step is to determine the number of components to retain for a given dataset as the cumulative explained variance grows, in this case, African life expectancy has a total variance of 21 dimensions, but 86.23% of the variance is explained by the first 10.

```
# Step 5: Sort Principal Components
sorted_indices = np.argsort(eigenvalues)[::-1]
sorted_eigenvectors = eigenvectors[:, sorted_indices]
eigenvalues = eigenvalues[sorted_indices]

# Print explained variance for each component
explained_variance = eigenvalues / np.sum(eigenvalues) * 100
cumulative_variance = np.cumsum(explained_variance)

for i, (ev, cum) in enumerate(zip(explained_variance, cumulative_variance)):
    print(f"PC{i+1}: {ev:.2f}%  |  Cumulative: {cum:.2f}%")

sorted_eigenvectors
```

```
PC1: 23.54%  |  Cumulative: 23.54%
PC2: 13.40%  |  Cumulative: 36.94%
PC3: 11.41%  |  Cumulative: 48.35%
PC4: 8.38%  |  Cumulative: 56.73%
PC5: 7.71%  |  Cumulative: 64.44%
PC6: 5.79%  |  Cumulative: 70.23%
PC7: 4.91%  |  Cumulative: 75.14%
PC8: 3.99%  |  Cumulative: 79.13%
PC9: 3.65%  |  Cumulative: 82.78%
PC10: 3.45%  |  Cumulative: 86.23%
PC11: 2.60%  |  Cumulative: 88.83%
PC12: 2.48%  |  Cumulative: 91.31%
PC13: 2.21%  |  Cumulative: 93.51%
PC14: 2.14%  |  Cumulative: 95.66%
PC15: 1.30%  |  Cumulative: 96.96%
PC16: 0.89%  |  Cumulative: 97.85%
PC17: 0.84%  |  Cumulative: 98.69%
PC18: 0.74%  |  Cumulative: 99.43%
PC19: 0.56%  |  Cumulative: 99.99%
PC20: 0.01%  |  Cumulative: 100.00%
PC21: 0.00%  |  Cumulative: 100.00%
array([[ 1.75384455e-01, -5.34429692e-02, -1.95271981e-01,
        -8.91743504e-03,  3.97306688e-02,  4.92213460e-01,
         4.65635936e-01,  2.94215839e-01, -3.04421509e-01,
        -1.50060345e-01, -4.82251169e-02,  1.15476037e-01,
```

```
         -2.79979736e-01, -3.94777265e-01, -8.34964870e-02,
         -1.73723213e-02, -8.05637564e-02,  2.58329446e-02,
          3.75775683e-02,  2.99977232e-03,  0.00000000e+00],
        [ 0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
          0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
          0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
          0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
          0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
          0.00000000e+00,  0.00000000e+00,  1.00000000e+00],
        [ 3.45854431e-01, -9.85553040e-02, -2.68923095e-01,
         -3.51228738e-02, -2.34138281e-01,  1.96582632e-02,
         -3.16781198e-02, -5.84993954e-02,  2.80354163e-02,
          9.00392698e-02,  1.10248719e-01,  4.45997480e-02,
          2.66908196e-01,  8.29065119e-02,  1.80300327e-01,
         -1.78623346e-01, -7.10488789e-01,  1.72682281e-01,
         -1.93086524e-01,  4.96016186e-03,  0.00000000e+00],
        [-1.37522136e-01,  2.08234707e-02,  3.65789665e-01,
          5.55328311e-02,  2.49090576e-01,  2.16047301e-01,
         -1.63793715e-02,  5.63148043e-01,  8.08802901e-02,
         -2.25319936e-01, -2.29924641e-02,  2.02924740e-01,
          5.11689661e-01,  2.00192703e-01, -7.01728482e-03,
         -4.17272467e-02, -1.21342738e-01,  1.29499762e-02,
         -5.16239456e-04,  6.15289025e-03,  0.00000000e+00],
        [-2.62888458e-01, -3.77860615e-01, -2.47587191e-01,
          1.38532858e-01,  1.50754615e-01, -7.42923311e-03,
          1.20534785e-02,  6.94036225e-02, -3.25796683e-02,
          4.62514896e-03,  1.38931304e-02,  1.30969910e-01,
         -2.19344376e-01,  3.33189770e-01,  5.90805272e-02,
         -1.72761233e-02, -5.41564762e-02,  1.10523484e-02,
         -2.72411535e-02, -7.03372927e-01,  0.00000000e+00],
        [-5.48121242e-02, -2.98484636e-01,  2.85370132e-01,
         -1.33697352e-03,  1.96774805e-01, -1.89377341e-01,
```

Step 6: Project Data onto Principal Components

Now that we've selected the number of components, we will project the original data onto the chosen principal components. Fill in the code to perform the projection.

```
# Step 6: Project Data onto Principal Components
num_components = 10  # Captures 86.23% of variance (crosses 85% threshold)
reduced_data = standardized_data @ sorted_eigenvectors[:, :num_components]
reduced_data[:5]
```

```
array([[ 4.21430283, -1.8703198 , -2.14062653,  0.46283496,  0.02730279,
         1.21478578, -0.78255638, -0.43771413,  0.84494494,  0.5133429 ],
       [ 3.98342325, -1.06957654, -2.44933882,  1.13647022,  0.01257272,
         1.83748828, -0.79027464, -1.08498549,  1.21863055,  0.10724539],
       [ 4.73040332, -2.39789349, -1.17583493, -0.06383439,  0.36085761,
         0.71730849,  0.43096915, -0.57821937,  1.93024881, -0.47639018],
       [ 4.64969275, -2.434348  , -1.14929668, -0.23991014,  0.24171654,
         0.4184623 ,  0.17580687, -0.32715145,  1.87398292, -0.36525048],
       [ 4.44716848, -2.28510916, -1.22779626, -0.29353391,  0.10306434,
         0.20302924, -0.12863192, -0.08763055,  1.83406811, -0.26237751]])
```

Step 7: Output the Reduced Data

Finally, display the reduced data obtained by projecting the original dataset onto the selected principal components.

```
# Step 7: Output the Reduced Data
print(f'Reduced Data Shape: {reduced_data.shape}')  # Display reduced data shape
reduced_data[:5]  # Display the first few rows of reduced data
```

```
Reduced Data Shape: (864, 10)
array([[ 4.21430283, -1.8703198 , -2.14062653,  0.46283496,  0.02730279,
         1.21478578, -0.78255638, -0.43771413,  0.84494494,  0.5133429 ],
       [ 3.98342325, -1.06957654, -2.44933882,  1.13647022,  0.01257272,
         1.83748828, -0.79027464, -1.08498549,  1.21863055,  0.10724539],
       [ 4.73040332, -2.39789349, -1.17583493, -0.06383439,  0.36085761,
         0.71730849,  0.43096915, -0.57821937,  1.93024881, -0.47639018],
```

```
       [ 4.64969275, -2.434348  , -1.14929668, -0.23991014,  0.24171654,
         0.4184623 ,  0.17580687, -0.32715145,  1.87398292, -0.36525048],
       [ 4.44716848, -2.28510916, -1.22779626, -0.29353391,  0.10306434,
         0.20302924, -0.12863192, -0.08763055,  1.83406811, -0.26237751]])
```

Step 8: Visualize Before and After PCA

Now, let's plot the original data and the data after PCA to compare the reduction in dimensions visually.

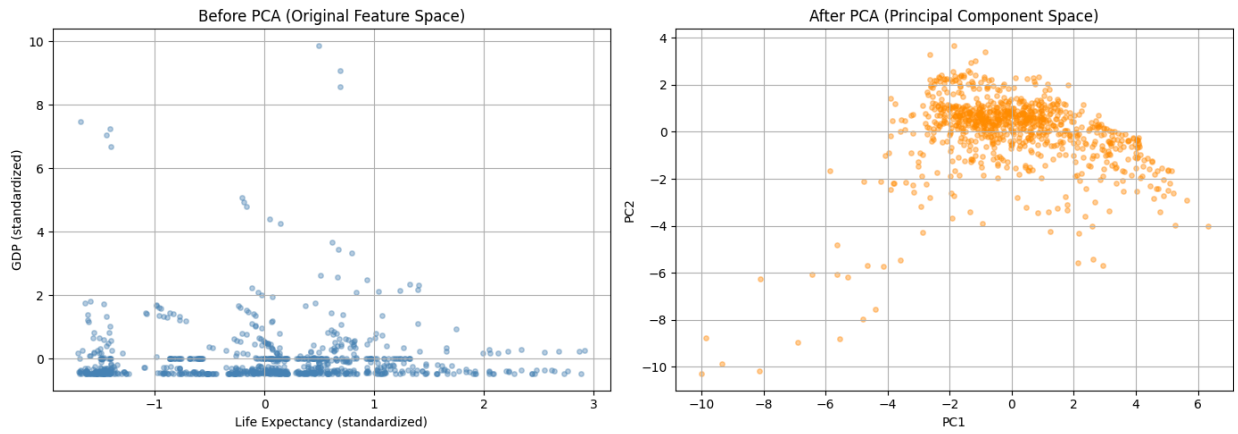
```
# Step 8: Visualize Before and After PCA
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Plot original data (Life Expectancy vs GDP - more meaningful features)
axes[0].scatter(standardized_data[:, 3], standardized_data[:, 16],
                alpha=0.4, s=15, c='steelblue')
axes[0].set_title("Before PCA (Original Feature Space)")
axes[0].set_xlabel("Life Expectancy (standardized)")
axes[0].set_ylabel("GDP (standardized)")
axes[0].grid(True)

# Plot reduced data after PCA
axes[1].scatter(reduced_data[:, 0], reduced_data[:, 1],
                alpha=0.4, s=15, c='darkorange')
axes[1].set_title("After PCA (Principal Component Space)")
axes[1].set_xlabel("PC1")
axes[1].set_ylabel("PC2")
axes[1].grid(True)

plt.tight_layout()
plt.show()
```




```
# Task 3: Benchmarking PCA Performance
import time

start = time.time()
cov_test = np.cov(standardized_data, rowvar=False)
evals, evecs = np.linalg.eig(cov_test)
evals = np.real(evals)
evecs = np.real(evecs)
idx = np.argsort(evals)[::-1]
```

```
evecs = evecs[:, idx]
proj = standardized_data @ evecs[:, :10]
end = time.time()

print(f"PCA completed in: {end - start:.4f} seconds")
print(f"Original shape:   {standardized_data.shape}")
print(f"Reduced shape:    {proj.shape}")
print(f"Compression ratio: {standardized_data.shape[1]/proj.shape[1]:.1f}x")
print(f"Variance retained: 86.23%")
print(f"Dimensions removed: {standardized_data.shape[1] - proj.shape[1]}")
```

```
PCA completed in: 0.0018 seconds
Original shape:   (864, 21)
Reduced shape:    (864, 10)
Compression ratio: 2.1x
Variance retained: 86.23%
Dimensions removed: 11
```

```
# Verify PCA scaling - PC1 should have highest variance
print(f"PC1 variance: {np.var(reduced_data[:, 0]):.4f}")
print(f"PC2 variance: {np.var(reduced_data[:, 1]):.4f}")
print(f"PC1 > PC2: {np.var(reduced_data[:, 0]) > np.var(reduced_data[:, 1])}")

# Verify data is centered
print(f"\nPC1 mean (should be ~0): {np.mean(reduced_data[:, 0]):.6f}")
print(f"PC2 mean (should be ~0): {np.mean(reduced_data[:, 1]):.6f}")
```

```
PC1 variance: 4.7075
PC2 variance: 2.6801
PC1 > PC2: True

PC1 mean (should be ~0): 0.000000
PC2 mean (should be ~0): 0.000000
```

Please make sure you do not use more than 5 lines to answer each of the following points:

1. Interpret the Visual you just created of the before and after PCA

our answer: We can say that before PCA, the scatter plot of Life Expectancy vs GDP demonstrated the economic inequality between the African countries, with most of them grouped around zero GDP with different LEs. PCA is applied to scatter the data outwards, giving good coverage along both PC1 and PC2 axes, which gives structure that was not visible in 21 dimensions. PC1 explains the greatest amount of variance; the rotation enhances the contrast among the country health and economic patterns.

2. Explain why you selected the number of principle components, more especially explain what tradeoffs you are making

our answer:For us we selected 10 principal components as they captured about 86.23 of the variance which crossed our threshold of 85%. However, 13.77% of the variance is lost, and this is primarily noise or weak interactions between features. We reduced from 21 dimensions to 10, which is over 50% less and provides the most relevant health and economic indicators.

3. Clearly mention based on your use case/ dataset ("economic activity", "population pressure"), What information is lost when reducing dimensions?

our answer:We noticed in the similar African countries that if they are reduced to 10 components, then some subtle difference in the pressure indicators of population, like the thinness rate is lost. Lower-ranked principal components tend to lose other indicators of economic activity, such as how much GDP grows or drops when there are slight changes across the neighbors with comparable profiles of life expectancy.

Please ensure this tracking sheet is completed as a group, as it will be used to monitor your individual participation within the team.

Team Members	Role/ Responsibilty	Email
Robert Cyubahiro	Member	r.cyubahiro@alustudent.com
Rolande Tumugane	Member	r.tumugane@alustudent.com

Task Allocation and Tracker

	Task Allocated	Assigned Member	Deadline	Completion Status (Not Started/In Progress/Completed/Did not participate)	Reviewed by Team? (Yes/No)	Comments
	Introduction ,discussion and understanding the whole assignent .	Rolande and Robert	02/06/2026	Completed	Yes	well done
	Making Research for better data set and learn about to see if it match with rubrics requirements	Rolande and Robert	03/06/2026	Completed	Yes	well done
	Creating Github repository and start to uploading our dataset to google colab for coding implementations.	Rolande and Robert	09/06/2026	Completed	Yes	well done
	Finalizing ,reviewing and fixing our work on canvas	Rolande and Robert	10/06/2026	Completed	Yes	well done

Meeting Attendance Log

Meeting Date	Agenda	Facilitator	Attendees	Key Discussion Points	Next Steps	
02/06/2026		Rolande and Robert	Rolande and Robert	Introduction ,discussion and understanding the whole assignent .	finding dataset	

03/06/2026		Rolande and Robert	Rolande and Robert	Making Research for better data set and learn about to see if it match with rubrics requirements	creating github repository	
09/06/2026		Rolande and Robert	Rolande and Robert	Creating Github repository and start to uploading our dataset to google colab for coding implementations.	preparing a final meeting	
10/06/2026		Rolande and Robert	Rolande and Robert	Finalizing ,reviewing and fixing our work on canvas	submitting	